

Local-GS: An Order-Independent Gaussian Splatting Training Accelerator Exploiting Splat Locality

Yiyang Sun, Qinzhe Zhi, Yiqi Jing, Le Ye, Ru Huang and Tianyu Jia*

School of Integrated Circuits, Peking University, Beijing, China

*Corresponding Email: tianyu.jia@pku.edu.cn

Abstract—3D Gaussian Splatting has emerged as the SOTA approach for 3D representation and view synthesis. While Gaussian Splatting has demonstrated impressive capability and rendering quality on desktop GPUs, achieving on-demand training on resource-constrained edge devices is still challenging. In this work, we identified the training bottleneck from a few perspectives including algorithm splat locality and the limited memory and hardware under-utilization. To address these problems, we present Local-GS, a 3D Gaussian Splatting training accelerator with order-independent rendering to break the depth-wise data dependency between overlapping Gaussians. We further incorporate a parallel pixel intersection test unit to schedule thread workload based on Gaussian splat locality and improve hardware utilization. A set of unified training-rendering cores are designed to achieve efficient splat-level parallel rendering and gradient propagation. Our Local-GS is implemented in 7nm and is evaluated by several real-world 3D scenes. Compared to edge Jetson NX GPU, Local-GS achieve 26.9-53 $\times$ training speedup and three orders of magnitude efficiency boost.

Index Terms—Gaussian Splatting, 3D, Training, Accelerator

I. INTRODUCTION

Novel view synthesis (NVS) is a widely studied computer vision task to generate photorealistic images from unobserved locations for a given 3D scene. Such generation tasks are highly desired in many consumer-level edge applications to provide a real-time seamless experience, such as AR/VR [1], live-streaming [2], and autonomous driving [3]. Traditional NVS algorithms [4] perform image synthesis based on 3D representations of meshes, voxels, or point clouds and leverage rasterization and ray tracing to generate object models, followed by output synthesized views. However, the conventional approach is computationally costly while it is challenging to achieve global consistency for high-quality rendering.

In recent years, learning-based NVS algorithms utilize implicit representation [5] and neural network models to learn and reconstruct 3D objects with remarkable quality. In particular, neural radiance field (NeRF) [6] is the most influential learning-based method that uses volume rendering with implicit neural scene representation to learn the geometry and lighting of a 3D scene. Although NeRF can offer a robust representation of 3D scene geometry, the challenges of high computational demand persist. 3D Gaussian splatting (3D GS) [7] is an emerging powerful technique for efficient and real-time NVS. Compared to NeRF, 3D GS is an emerging non-AI workload that portrays a 3D scene using numerous 3D Gaussian primitives or ellipsoids. Each ellipsoid in 3D GS is

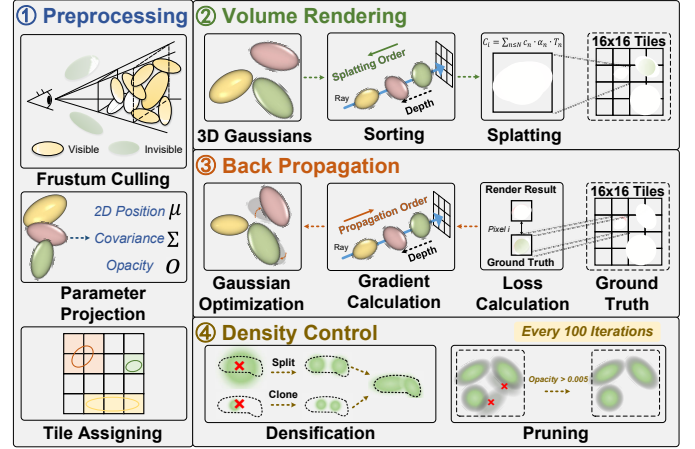


Fig. 1. Overview of 3D Gaussian Splatting operations.

equipped with position, orientation, scale, opacity, and color information. To render these ellipsoids, they undergo a conversion from 3D to 2D space and are strategically organized for optimal rendering. Compared to NeRF, 3D GS can achieve very fast convergence speed for training, e.g. 15-20 min for a 1M Gaussian scene on RTX 3090, and photorealistic NVS rendering in real time, e.g. 100 FPS on RTX 3090 GPU and 60 FPS on laptop RTX 3060 GPU.

Fig. 1 shows the main operation steps for 3D GS, including Gaussian preprocessing, volume rendering, loss backpropagation, and adaptive density control. Preprocessing transforms the 3D features of Gaussian ellipsoids into 2D, including frustum culling and tile assigning based on their scale and position. In volumetric rendering, the ellipsoids are “splatted” onto the camera plane in ascending order of depth to merge their 2D footprints into the rendered image and faithfully represent occlusion between transparent Gaussians. The back propagation training of GS begins with color loss calculation between ground truth images and rendered results, followed by a reversed rendering process, i.e. rendering Gaussians in the descending order of depth to peel off occlusion effects caused by former Gaussians. As for the ill-represented Gaussians with a large positional loss, 3D GS adopts a dynamic density control strategy, i.e. densification and pruning, to adjust the ellipsoid number and accurately represent the scene fine details.

Although 3D GS exhibits promising results for both training and rendering, it still faces considerable difficulty for the

deployment on mobile hardware. For instance, training a 1M Gaussian scene consumes 6 hours, and rendering a novel view will take 120-160 ms on edge platform Jetson Xavier NX. Considering that most NVS tasks are intended to be deployed on mobile devices, such lengthy training and rendering time is unacceptable for the applications.

Since 3D GS was first present in 2023, there are limited attempts to accelerate 3D GS via hardware implementations. For example, [8] presented a hardware acceleration unit to execute the rendering pipeline of 3D GS with algorithmic optimizations. [9] developed an accelerator for 3D GS-based SLAM rendering task. However, the existing works only focused on the rendering process, while the training is much more complicated for edge NVS deployment. In addition, the pre-trained NVS models often have poor versatility, i.e. render-only accelerators cannot make on-demand responses to the varying scenes. Hence the training capability at edge is critical to enable fast scene modeling to adapt to new environments for applications such as AR/VR and autonomous driving.

In this paper, we present Local-GS, a hardware-software co-designed accelerator for 3D GS training and rendering. Our work utilized the spatial locality of Gaussian splats to efficiently reschedule forward and backward rendering workload, as well as breaking the rendering order dependency to eliminate resource constraints on edge hardware. Implemented with 7nm technology, Local-GS achieves $26.9\text{-}53\times$ training speedup compared with Jetson NX and $3.0\text{-}4.8\times$ improvement compared to a RTX 3090 GPU across diverse scenarios. Significant energy efficiency boost is also obtained.

The contributions of this work are summarized as follows:

- We analyze the emerging 3D GS workload, and identify the memory and computation challenges due to the transparency dependencies in back propagation.
- We leverage a depth-guided order-independent blending scheme for parallel multi-Gaussian rendering and gradient propagation with significant performance gain.
- We develop a unified training-rendering digital architecture leveraging splat locality to achieve workload scheduling and high hardware utilization.
- We implement Local-GS accelerator using 7nm and evaluate it using a set of real-world benchmark, showing significant performance and efficiency gains.

II. BACKGROUND

A. Preliminaries of 3D Gaussian Splatting

3D Gaussian Splatting starts with a set of images to generate a sparse initial point cloud through structure from motion (SfM) [10]. As shown in Fig. 1, a set of ellipsoid-shaped Gaussian primitives is created based on these points, where each Gaussian ellipsoid is represented by a three-dimensional Gaussian distribution with position(μ), scale(s), rotation(r), opacity(o) and spherical harmonics (sh) for color representation. After initialization, 3D GS repetitively iterates through all camera sets to refine the 3D model with the following operation steps: preprocessing, volume rendering, back propagation and density control.

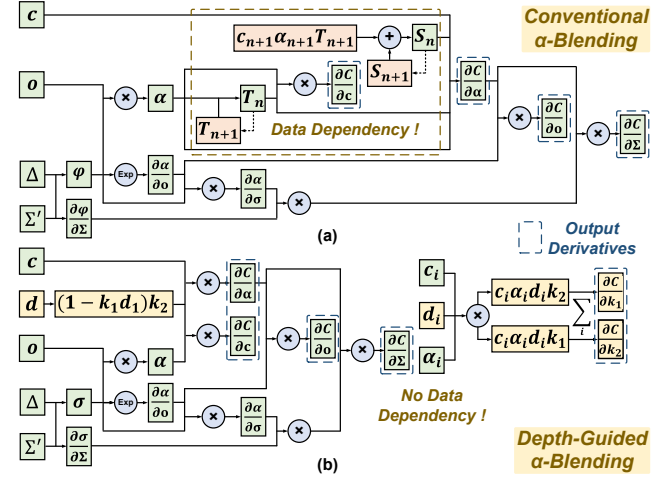


Fig. 2. Gradient computation in (a) conventional 3D-GS training, (b) 3D-GS training with depth-guided α blending.

Preprocessing of the given 3D Gaussian set starts by frustum culling the Gaussians of the scene outside the viewable area to focus on the Gaussians that need to be rendered. After obtaining visible Gaussians, the 3D features of Gaussians are converted into 2D position (μ), covariance matrix(Σ), opacity(o) on the projection plane. Meanwhile, a rectangular boundary is defined based on the projected Gaussian footprints to calculate their overlapping area with the screen. The Gaussians are then duplicated and assigned to the intersected image tiles (16×16 pixels) for tile-wise depth sorting to obtain correct color accumulation.

Volume rendering is carried out by tile-wise after preprocessing. The process begins by determining how much a Gaussian contributes to a certain pixel with its inherit transparency (α), and the accumulated transmittance (T) through the former-rendered Gaussians. The α value is calculated as following:

$$\alpha_i = o_i \cdot \exp\left(-\frac{1}{2}\Delta^T \Sigma^{-1} \Delta\right) = o_i \cdot \exp(\varphi) \quad (1)$$

where Δ is the position vector between the center of Gaussian and pixel and o_i representing Gaussian opacity. T is determined through $T_i = \Pi(1 - \alpha_i)$ across previously rendered Gaussians. The per-Gaussian contributions are accumulated, i.e. called α blending process, in ascending order of depth across all Gaussians in the tile to generate the pixel colors:

$$C = \Sigma T_i c_i \alpha_i \quad (2)$$

Back propagation optimizes the Gaussian distribution through the difference or loss (L) between rendered images and the ground truth images. 3D GS adopts direct L_1 loss and structural D-SSIM loss to measure the difference. The loss is then back-propagated to the Gaussian parameters with partial derivatives, as shown in Fig. 2(a). The partial derivatives related to c, α, o, Σ' are computed respectively with corresponding Gaussian parameters. However, in the computation graph of $\frac{\partial C}{\partial \alpha}$, the computation results of deeper Gaussians are also involved. More specifically, the accumulated transparency

T_n and reversely accumulated transparency S_n has data dependency upon T_{n+1} and S_{n+1} as Eq. 3.

$$T_n = \frac{T_{n+1}}{1 - \alpha_n}, S_n = c_{n+1} \alpha_{n+1} T_{n+1} + S_{n+1}. \quad (3)$$

Unlike the volume rendering process, the partial derivative related to α requires computation in the descending order of Gaussian depth due to the occlusion effects. Hence the Gaussians obtained during the rendering process should be stored to carry out reverse rendering process in back propagation.

Density control is executed every 100 iterations in the GS process to better model the unobserved scene details in the initial SfM point cloud. When density control is triggered, 3D GS detects the Gaussian ellipsoids with a large positional gradient, which indicates under or over reconstruction of scene detail. In such cases, either cloning or segmenting the Gaussian is implemented to densify the point cloud. As for the irrelevant transparent Gaussians, 3D GS periodically trims Gaussians with low opacity (α less than 0.005) to reduce the overall model size and improves rendering performance.

B. Order Independent Gaussian Splatting

As mentioned above, 3D GS requires depth-ordered Gaussian inputs to assure correct rendering results. However, sorting numerous Gaussians is costly, and limits the potential of intra-tile parallel rendering due to transmittance and gradient dependency. Recent algorithm advances start to explore partial-ordered [11] or order-independent [12] Gaussian splatting. For instance, depth guided weighted blending in [12] is an order-independent GS scheme, which completely break data dependency in rendering by using splat depth to guide the blending. By this way, the α blending is simplified as:

$$C = \sum c_i \cdot \alpha_i \cdot (1 - k_1 \cdot d_i) \cdot k_2 \quad (4)$$

in which d_i is the Gaussian depth and k_1, k_2 are learnable parameters. By replacing T with a depth-related factor, order-independent Gaussian splatting isolates the contribution of each Gaussian to pixel color and enables Gaussian-level parallelism. Moreover, the time cost to sort Gaussians and memory cost to store the ordered Gaussians list can be eliminated.

Fig. 2(b) shows the back propagation flow for depth-guided blending in order-independent GS. We mainly introduce the computation of $\frac{\partial C}{\partial c}$ and $\frac{\partial C}{\partial \alpha}$ as other partial derivatives can be calculated under the chain rule. Derived with Eq. 4, these two derivatives can be expressed as Eq. 5.

$$\frac{\partial C}{\partial c_n} = \alpha_n \cdot (1 - k_1 \cdot d_n) \cdot k_2, \quad \frac{\partial C}{\partial \alpha_n} = c_n \cdot (1 - k_1 \cdot d_n) \cdot k_2 \quad (5)$$

Similar to the volume rendering process, the partial derivatives of one certain Gaussian can be computed solely with its own parameters. Consequently, back propagation of multiple Gaussians can be simultaneously executed without order dependency when propagating the loss from a certain pixel. In conclusion, order-independent GS with depth guided weighted blending simplifies the computation and enables parallelism for both the volume rendering and back propagation.

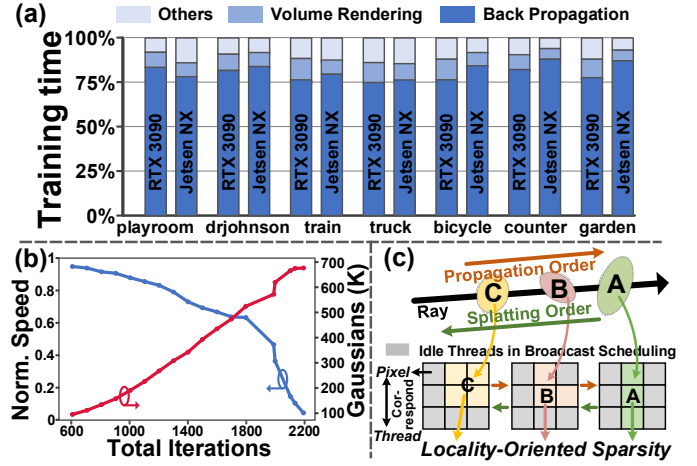


Fig. 3. (a) Runtime breakdown of RTX 3090(left) and Jetson NX(right). (b) throughput degradation caused by limited memory on Jetson NX. (c) thread idling and memory congestion in order-dependent Gaussian splatting.

III. WORKLOAD PROFILING AND ANALYSIS

To better understand the workload characteristics, we ran the end-to-end process of 3D GS on complex real-world scenes from Tanks & Templates, Deep Blending [13] and Mip-NeRF 360 [14] datasets. We ran these test sets on two representative hardware platforms, i.e. an RTX 3090 GPU and edge Nvidia Jetson Xavier NX platform. A total of 30000 training iterations are sampled on RTX 3090, while 3000-7000 iterations are run on Jetson NX due to memory limitation.

As the runtime breakdown shown in Fig. 3(a), similar runtime distribution is observed on both platforms. We identify the back propagation process as the main bottleneck which contributes more than 70% of total runtime, while the volume rendering process consumes only around 10% latency. This is partially because of the numerous gradient update and propagation introduced in the backward propagation process. The remaining latency comes from other processes, including the preprocessing and density control process. When we analyze the training process on Jetson NX, we noticed a significant throughput degradation during training (Fig. 3(b)), since the growing Gaussian primitive numbers cause the memory out of space for the sorted Gaussian lists and rendering features.

Apart from memory limitation, we also observe another key reason for the lengthy back propagation, i.e. severe hardware under-utilization caused by locality-oriented Gaussian sparsity. As shown in Fig. 3(c), in conventional 3D GS flow, the target Gaussian splat is broadcast to the a thread block in GPU and each thread is assigned to render a certain pixel. However, in reality the Gaussian ellipsoids tend to be small to faithfully represent surface details, i.e. only a few pixels *local* to the splat center will be rendered. Such phenomenon leaves considerable pixel threads idle, which we refer as *Locality-Oriented Sparsity* or *Splat Locality*. In a typical thread block, splat locality will cause around 80-90% threads stay idle, being one major bottleneck in 3D GS.

To address the above limitations, we explore an efficient 3D GS training accelerator solution with the following strategies. First, to overcome memory limitations on mobile platforms,

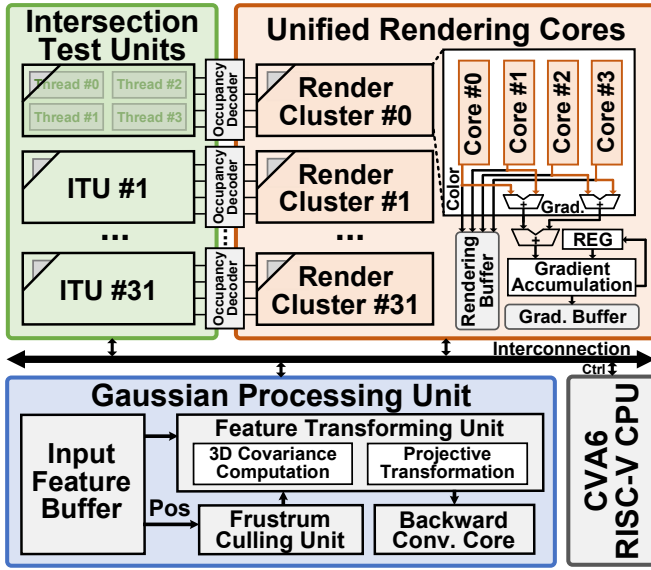


Fig. 4. Architecture overview of our Local-GS training accelerator.

we adopt an order-independent Gaussian rendering scheme to eliminate sorting latency and storage overhead. Second, to reduce the thread idling caused by splat locality, we employ pixel-wise intersection test and Gaussian-centered workload scheduling scheme. Hence the rendering threads only need to focus on pixels that intersected with the local sphere of GS.

IV. TRAINING ACCELERATOR FOR 3D GS: LOCAL-GS

A. Architecture Overview

In this section, we introduce Local-GS, a unified training-rendering accelerator for order-independent 3D GS exploiting the splat locality feature. As shown in Fig. 4, Local-GS mainly consists of three components: a Gaussian processing unit that transforms 3D Gaussian features into 2D for splat rendering, 32 clusters of unified rendering cores for parallel volume rendering and back propagation and 32 intersection test units (ITUs) paired with rendering clusters through an occupancy decoder for parallel Gaussian intersection testing in the local regions. A host RISC-V CPU core is added for programming and general-purpose status control.

During 3D GS operations, the Gaussian processing unit first culls out redundant Gaussians for image rendering. Following that, the feature transforming unit converts the 3D Gaussian features into 2D features, as well as transforming the coordinate base vectors to assist pixel-wise intersection test. The Gaussian processing unit also holds a tiny convolution core to calculate simplified SSIM loss with reduced kernel size for backward propagation. With the help of base vectors, the ITUs perform incremental pixel-level intersection test local to the gaussian center to generate sparse occupancy bitmaps. The compressed bitmaps are then decoded into continuous pixel flow for rendering core workload assignment. Finally, the Gaussian rendering cores execute the forward volume rendering, or backward loss propagation, or Gaussian gradients.

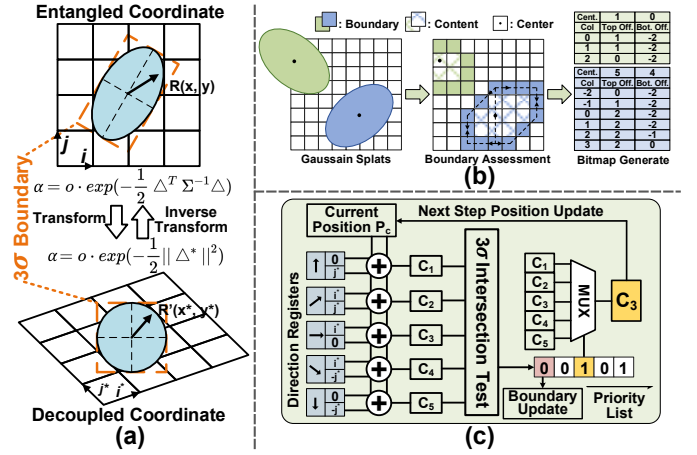


Fig. 5. (a) Our proposed intersection test method, (b) the workload scheduling of an ITU, and (c) detailed architecture of an ITU thread.

The Gaussian processing unit consists of a frustum culling unit (FCU) to cull out the out-of-view Gaussians, a feature transforming unit (FTU) to process Gaussian features and coordinate base vectors for intersection testing and a small convolution core for loss calculation. During the preprocessing stage, the FCU first identifies the viewing frustum boundaries as unit length, then normalize the input Gaussian coordinates accordingly. The FTU converts the 3D features of Gaussian ellipsoids into 2D features on the projection plane through estimated affine transformation with local Jacobian matrix to calculate Gaussian center position(μ), opacity(o), transformation matrix(T), and obtain an RGB color(c).

B. Parallel Intersection Test Unit

In this work, we adopted a local Gaussian-centered pixel-wise parallel intersection test to merge bounding-box detection and α computation at pixel-level, leveraging the splat locality feature and enhance the hardware utilization. In the original Gaussian splatting, intersection test creates a rough boundary for each GSussian splat. The contribution of pixels inside the boundary is measured by α , which is calculated pixel wise with precise positional information.

Fig. 5(a) exhibits our intersection test, in which an incremental α value computation of candidate pixels is performed once pixel in the 3σ local Gaussian boundary. By re-organizing covariance calculation in Eq. 1, the non-diagonal inverse covariance matrix Σ^{-1} can be represented with the 2D transformation $\Sigma^{-1} = (TT^T)^{-1}$ and the α entangled with X, Y can be decoupled as a 2D Gaussian kernel,

$$\alpha = o \cdot \exp\left(-\frac{1}{2}(T^{-1}\Delta)^T(T^{-1}\Delta)\right) = o \cdot G(\Delta^*) \quad (6)$$

where $\Delta^* = T^{-1}\Delta$, or two component 1D Gaussian kernels along major and minor axis of Gaussian ellipsoid:

$$\alpha_i = o_i \cdot G(\Delta_x^*) \cdot G(\Delta_y^*) \quad (7)$$

With this transform, we can incrementally iterate through pixels and identify their α values through Δ^*

$$\Delta^*(m, n) = \Delta^* + m \cdot i^* + n \cdot j^* \quad (8)$$

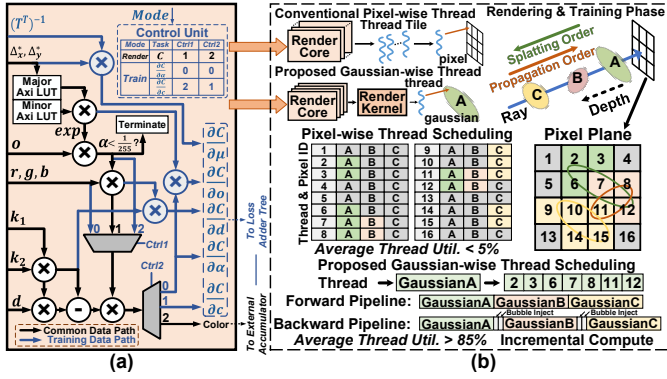


Fig. 6. (a) The training/rendering datapath of our Unified Rendering Core, (b) comparison between our Gaussian-centered workload scheduling and conventional scheduling in the original 3D GS.

using relative pixel position m, n and transformed base vectors $i^* = T^{-1}i$ and $j^* = T^{-1}j$.

Fig. 5(b) illustrates the workflow of an ITU, where four working threads collaboratively search for intersected pixels within the local Gaussian splat. The process begins at the Gaussian center, and threads search along the Y axis until reaching the splat boundary, identifying valid top and bottom boundary pixels. The threads then split into two groups to explore adjacent columns along the X axis. The search terminates when the top and bottom pixels of the same column overlap, with no more adjacent intersected tiles. During the searching process, the ITU registers the offset of top and bottom from Gaussian center pixels of each column, i.e. 1 and -2 for column -1 of the blue splat, as well as the original pixel ID of the Gaussian center, i.e. (5,4), into a compact occupancy table to efficiently store the locality-based sparse intersection bitmap and schedule rendering core workload accordingly.

The detailed thread architecture of the ITU is shown in Fig. 5(c). When a thread reaches a new pixel, the working thread examines the valid searching directions and combines the corresponding transformed base vectors i^*, j^* stored in direction registers with current pixel position to generate next step position candidates. The candidate positions are then processed through the 3σ intersection test to check whether they are inside the Gaussian's local boundary. A candidate will be considered active only when both component vectors are located inside the boundary. The next step position is then updated according to the active candidate with highest priority, and update the ITU boundary register if the top-priority candidate C_1 is inactive.

C. Local-GS Rendering Core

Fig. 6(a) shows the datapath design of our unified rendering core. The core can render Gaussians or propagate partial gradients according to the assigned mode signals. In volume rendering mode, the training data path stays inactive, and the input Gaussian parameters flow through the common data path to generate rendered pixel color. The rendering core calculates the pixel α value as Eq. 7. Following that, α is weighted by Gaussian depth to formulate a blending component as Eq. 4.

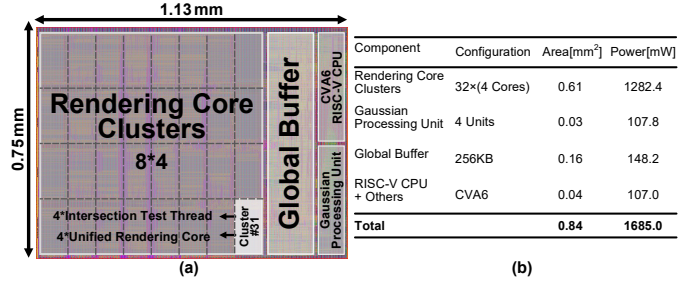


Fig. 7. (a) Layout view of Local-GS accelerator in 7nm technology, (b) design specification of Local-GS.

In back propagation mode, the training data path will be activated together with the common data path to calculate partial gradients in Fig. 2(b). The output gradients will be directed to a local adder tree in the kernel to accumulate with gradients obtained in other threads to form the accumulated splat-level gradient for parameter optimization. With local gradient reduction, pipeline congestion caused by simultaneous gradient updates can be eliminated.

Fig. 6(b) illustrates the workload scheduling strategy for rendering cores. In conventional pixel-wise thread scheduling in GPU and accelerator [8], the hardware utilization is low due to the splat locality feature, e.g. Gaussian A, B, and C only affect 7, 4, and 5 pixels in the pixel plane. Hence we adopted a Gaussian-centered scheduling scheme to improve hardware utilization. We aggregate four rendering cores to form a rendering kernel, which corresponds to one thread and processes a Gaussian. The thread calculates four pixels affected by the current Gaussian every cycle. When the thread completes the forward rendering for current Gaussian, changing assigned Gaussians will not influence the pipeline, as the inputs are pixel-aligned and will be accumulated upon storage. While in the backward running, we inject two pipeline bubbles between Gaussians to ensure the local gradient adder tree has finished the accumulation and generated valid output results. With all factors considered, our proposed workload scheduling scheme achieved more than 85% hardware utilization on average, which greatly outperformed the baseline method.

V. EVALUATIONS

A. Implementation

We implemented our Local-GS accelerator using a 7nm technology [15], as the layout view shown in Fig. 7. The accelerator is fully developed in RTL, with the I/O buffers and CPU main memory generated by a commercial SRAM compiler. The design is synthesized by Cadence Genus and physically implemented by Cadence Innovus at 1GHz. The total active area of Local-GS is 1.13×0.75 mm for 32 rendering core clusters and host CPU, for which we adopt an open-source RISC-V CVA6 CPU [16]. The accelerator area is dominated by the 32 rendering core clusters, i.e. 72.7%, and corresponding buffers, i.e. 18.9%. The detailed design breakdown of Local-GS is listed in Fig. 7(b). The following energy, performance, and area results are reported based on post-P&R netlist at the typical corner, i.e. 0.7V at 25°C.

Scene	Playroom	Drjohnson	Train	Truck	Bicycle	Counter	Garden
3DGS	37.11	34.57	25.11	29.01	25.12	29.19	27.32
Sort-free	37.19	35.26	25.04	29.44	24.53	29.57	27.56
Local-GS	37.03	35.11	24.83	28.87	24.40	29.21	27.17

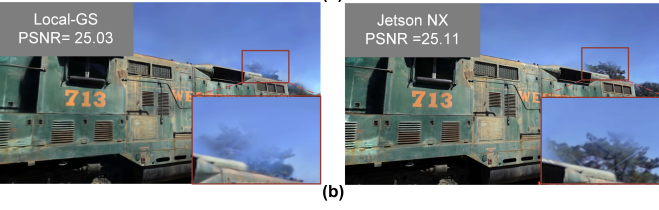


Fig. 8. (a) PSNR evaluation results, (b) example of one rendered image.

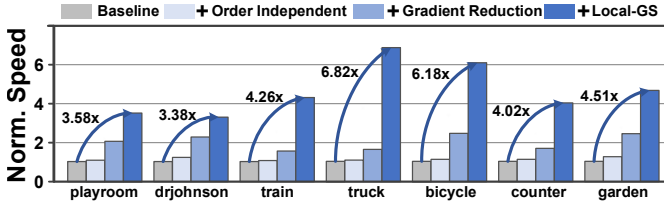


Fig. 9. Normalized speedup for 3D GS with our Local-GS accelerator.

We evaluate our order-independent architecture by comparing with the original 3D GS [7] and Sort-Free GS [12], which adopts the depth-guided blending method. Our Local-GS is compared with existing hardware of Jetson NX and RTX 3090 GPU using real-world workloads from multiple datasets with both indoor and outdoor scenes. The datasets include Tank and Templates, Deep Blending, and Mip-NeRF 360 dataset.

B. Benefits Evaluations

In this section, we evaluate the performance and efficiency benefits of our Local-GS accelerator. First, we compare our rendered image quality with the original 3D GS and Sort-Free GS. Fig. 8 presents the PSNR score across multiple rendering categories and an example rendering result from the train dataset. Local-GS achieves almost the same rendering quality compared with 3D GS and Sort-Free GS on the train dataset, i.e. 24.83 compared with 25.11 and 25.04. The degradation is mainly invoked by the order independent rendering process, which results in the semi-transparent object in Fig. 8(b).

To better evaluate the efficiency of our architecture, we compare the normalized speedup with state-of-the-art Gaussian rendering accelerator, GScore [8]. Since GScore only supports forward rendering, we provided it with a training hardware plugin of the same architecture and specifications as the baseline to ensure a fair comparison. As shown in Fig. 9, having optimizations of order-independent rendering and gradient loss reduction on GScore improve the performance by an average of $1.1\times$ and $1.8\times$, respectively. With our Local-GS architecture, an average $4.7\times$ speedup is obtained across different scenarios compared with GScore, which is mainly contributed by leveraging the splat locality via the pixel-wise intersection test and Gaussian-centered workload scheduling. For particular 3D scene category, Local-GS performs best on Truck and Bicycle scenes, yielding $6.82\times$ and $6.18\times$ speed-up compared to GScore. This is mainly because of

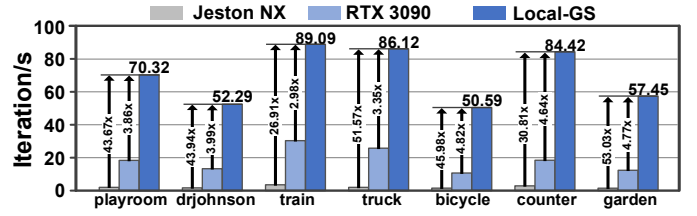


Fig. 10. 3D GS training throughput of baseline hardware and our design.

TABLE I
COMPARISON BETWEEN LOCAL-GS, GSCORE AND JETSON NX

	Ours	GScore	Xavier NX
Frequency	1GHz	1GHz	1.1GHz
core Num.	32*4	4*16	384 CUDA Cores
Technology	7nm	28nm	12nm
Precision	FP16	FP16	FP32
Power(mW)	1685	870	2×10^4
Area(mm ²)	0.848(Post P&R)	3.95(Synthesize)	350
Energy Efficiency(mJ/Iter)	19.6	9.88(Rendering Only)	6.06×10^3
Support Training	✓	✗	✓
Training Time	5-10min	✗	4-6hour
Rendering Speed (FPS)	350.6	79.5	10.6

the frequent sudden depth changes that disabled the efficient early termination scheme in GScore. In contrast, indoor scenes are relatively challenging, where objects may be completely occluded from sight, e.g., $3.58\times$ and $3.38\times$ speed-up for Playroom and Drjohnson.

C. Comparison and Discussion

Fig. 10 shows our comparison with existing GPU hardware. Thanks to the benefits of software-hardware co-optimizations, Local-GS achieved a training throughput improvement by $26.9\text{--}53\times$ across different scenarios compared to edge Jetson NX. Compared to the high-performance RTX 3090 GPU, Local-GS also receives an improvement of $3.0\text{--}4.8\times$.

Table I lists the detailed comparison between our Local-GS accelerator and prior accelerator and edge GPU. Compared with available training-capable edge hardware, e.g. Jetson NX, Local-GS achieves about $50\times$ throughput gain and a 19.6 mJ/iteration efficiency, which is a nearly three orders of magnitude efficiency boost. For rendering, Local-GS achieves $4.38\times$ better throughput than dedicated rendering GScore since we exploit splat locality and obtain an average 85% hardware utilization. Overall, our Local-GS provides a promising 3D training/rendering hardware solution, which can be applied for diverse on-demand real-time NVS applications.

VI. CONCLUSION

In this paper, we presented a unified training-rendering accelerator, Local-GS, to facilitate on demand scene training and real-time rendering on edge devices. Compared to other accelerators, Local-GS achieves $3.5\times$ hardware utilization increase by exploiting localization-based Gaussian sparsity and leveraging order-independent parallel training-rendering scheme. Our Local-GS accelerator is implemented and evaluated in 7nm technology, achieving $26.9\text{--}53\times$ faster training speed and three orders of magnitude energy efficiency boost compared with Jetson Xavier NX mobile GPU.

REFERENCES

- [1] J. Dai, Z. Zhang, S. Mao, and D. Liu, "A view synthesis-based 360° vr caching system over mec-enabled c-ran," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 30, no. 10, pp. 3843–3855, 2020.
- [2] S. Lewin, M. Vandegar, T. Hoyoux, O. Barnich, and G. Louppe, "Dynamic nerfs for soccer scenes," in *6th International Workshop on Multimedia Content Analysis in Sports*, p. 113–121, 2023.
- [3] X. Zhou, Z. Lin, X. Shan, Y. Wang, D. Sun, and M.-H. Yang, "Drivinggaussian: Composite gaussian splatting for surrounding dynamic autonomous driving scenes," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 21634–21643, June 2024.
- [4] A. Kar, C. Häne, and J. Malik, "Learning a multi-view stereo machine," in *31st International Conference on Neural Information Processing Systems (NIPS)*, 2017.
- [5] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, "Deepsdf: Learning continuous signed distance functions for shape representation," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 165–174, 2019.
- [6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, p. 99–106, dec 2021.
- [7] B. Kerbl, G. Kopanas, T. Leimkuehler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, jul 2023.
- [8] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "Gscore: Efficient radiance field rendering via architectural support for 3d gaussian splatting," in *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 497–511, 2024.
- [9] L. Wu, H. Zhu, S. He, J. Zheng, C. Chen, and X. Zeng, "Gauspu: 3d gaussian splatting processor for real-time slam systems," in *57th IEEE/ACM International Symposium on Microarchitecture (MICR)*, 2024.
- [10] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4104–4113, 2016.
- [11] F. Hahlbohm, F. Friederichs, T. Weyrich, L. Franke, M. Kappel, S. Castillo, M. Stamminger, M. Eisemann, and M. Magnor, "Efficient perspective-correct 3d gaussian splatting using hybrid transparency," 2024.
- [12] Q. Hou, R. Rauwendaal, Z. Li, H. Le, F. Farhadzadeh, F. Porikli, A. Bourd, and A. Said, "Sort-free gaussian splatting via weighted sum rendering," 2024.
- [13] P. Hedman, J. Philip, T. Price, J.-M. Frahm, G. Drettakis, and G. Brostow, "Deep blending for free-viewpoint image-based rendering," *ACM Trans. Graph.*, vol. 37, dec 2018.
- [14] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-nerf 360: Unbounded anti-aliased neural radiance fields," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5460–5469, 2022.
- [15] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "Asap7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016.
- [16] B. Sá, L. Valente, J. Martins, D. Rossi, L. Benini, and S. Pinto, "Cva6 risc-v virtualization: Architecture, microarchitecture, and design space exploration," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2023.